

Ambiente Técnico: CalcEngine

Author: Raja SP, AWS Labs

Source: <https://github.com/awslabs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [example-minimal-tech-env-scientific-calculator-api.pt-br.pdf](#)

Linguagem e Gerenciador de Pacotes

- **Python 3.12+**
- **uv** para todo o gerenciamento de pacotes (sem pip, poetry ou conda)
- `pyproject.toml` para toda configuração de projeto e ferramentas
- `uv.lock` commitado no Git

Framework Web

- **FastAPI** com Pydantic v2 para validação de requisições/respostas
- **Mangum** para executar FastAPI no AWS Lambda

Cloud e Deploy

- **AWS**, conta única, `us-east-1`
- **Serverless:** Lambda atrás do API Gateway (tipo HTTP API)
- **DynamoDB** para armazenamento de chaves de API e medição de uso
- **S3 + CloudFront** para o site de documentação
- **AWS CDK (Python)** para toda a infraestrutura — sem mudanças manuais no console

Testes

- **pytest** com pytest-cov (mínimo de 90% de cobertura de linhas)
- **hypothesis** para testes de precisão matemática baseados em propriedades
- **mypy** modo strict para verificação de tipos

- **ruff** para linting e formatação
- **moto** para mockar serviços AWS nos testes

NÃO Use

Proibido	Motivo	Use ao Invés Disso
<code>eval()</code> , <code>exec()</code> , <code>compile()</code>	Segurança -- execução de código arbitrário	Parser de expressão baseado em AST
Flask, Django	O projeto usa FastAPI	FastAPI
requests	Bloqueia o event loop assíncrono	httpx
sympy	Muito pesado para o MVP	Parser de expressão customizado
pandas	Desnecessário -- cálculos únicos, não dataframes	Python padrão
pip, poetry, pipenv	O projeto usa uv exclusivamente	uv
black, flake8, isort	Substituídos por ruff	ruff
AWS EC2, ECS, RDS	Modelo serverless preferido para o MVP	Lambda, DynamoDB

Requisitos Básicos de Segurança

- Autenticação por chave de API via cabeçalho `Authorization: Bearer {key}`
- Chaves armazenadas como hashes bcrypt no DynamoDB, nunca registradas em texto simples
- O parser de expressão usa uma allowlist de caracteres e avaliação AST — sem execução de código dinâmico
- Comprimento de expressão limitado a 4.096 caracteres, profundidade de aninhamento limitada a 100 níveis
- TLS 1.2+ obrigatório, sem endpoints HTTP
- Segredos no AWS Secrets Manager, não em variáveis de ambiente ou código

Padrão de Código de Exemplo

Um endpoint deve seguir esta estrutura:

```
from fastapi import APIRouter, Depends
from pydantic import BaseModel, Field

from calcengine.api.middleware.auth import get_api_key_id
from calcengine.api.models.errors import error_response
from calcengine.api.models.responses import CalculationResponse
from calcengine.engine.errors import CalcEngineError
from calcengine.engine.trigonometry import sin

router = APIRouter()

class SinRequest(BaseModel):
    value: float
    angle_mode: str = Field(default="radians", pattern="^(radians|degrees)$")

@router.post("/v1/trigonometry/sin", response_model=CalculationResponse)
async def calculate_sin(
    request: SinRequest,
    api_key_id: str = Depends(get_api_key_id),
) -> CalculationResponse | dict:
    try:
        result = sin(request.value, angle_mode=request.angle_mode)
        return CalculationResponse(result=result, expression=f"sin({request.value})")
    except CalcEngineError as e:
        return error_response(e)
```

Uma função matemática deve seguir esta estrutura:

```

import math

from calcengine.engine.errors import DomainError

def log_base(value: float, base: float = 10.0) -> float:
    """Compute logarithm of value with given base. Raises DomainError for invalid input."""
    if value <= 0:
        raise DomainError(
            code="DOMAIN_ERROR",
            message=f"Cannot compute logarithm of {value}",
            detail="Logarithm is only defined for positive numbers",
        )
    if base <= 0 or base == 1.0:
        raise DomainError(
            code="DOMAIN_ERROR",
            message=f"Invalid logarithm base: {base}",
            detail="Base must be positive and not equal to 1",
        )
    return math.log(value) / math.log(base)

```

Um teste deve seguir esta estrutura:

```

import math
import pytest
from hypothesis import given, strategies as st
from calcengine.engine.errors import DomainError
from calcengine.engine.logarithmic import log_base

def test_log10_of_100() -> None:
    assert log_base(100, 10) == pytest.approx(2.0)

def test_log_of_negative_raises_domain_error() -> None:
    with pytest.raises(DomainError):
        log_base(-5)

@given(st.floats(min_value=1e-300, max_value=1e300, allow_nan=False, allow_infinity=False))
def test_log10_matches_stdlib(x: float) -> None:
    assert log_base(x, 10) == pytest.approx(math.log10(x), rel=1e-14)

```